



Assignment - AI Engineer Role

AI Threat Hunting Query Generation & Evaluation System

Overview

You are building an AI-powered query generation system that translates threat hunting hypotheses into executable queries against security log datasets. The system should generate accurate queries from natural language hypotheses and provide a robust evaluation framework to measure query quality and result accuracy.

This assignment uses the public CloudTrail dataset (available on Kaggle). You will be provided with pre-defined threat hypotheses and their expected outcomes - your job is to build the query generator and prove it works through comprehensive evaluation.

Goals

1. Build a query generation system that can translate natural language threat hunting hypotheses into executable queries (SQL, pandas operations, or any queryable interface of your choice).
2. Implement an evaluation framework with appropriate metrics.
3. Provide explainable outputs - the system should show its reasoning: how it interpreted the hypothesis, why it structured the query that way, and how results map to expected findings.



Provided Resources

1. Datasets

- CloudTrail dataset - Public dataset from flaws.cloud, mirrored on [Kaggle](#)
 - Download and load this into your preferred format (CSV, Parquet, DuckDB, SQLite, pandas DataFrame, etc.)
 - **Important:** Use **nineteenFeaturesDf.csv** file from the dataset

2. Hypotheses & Expected Outcomes

- `hypotheses.json` - Contains 15-20 threat hunting hypotheses in natural language
- `hypotheses_outcomes.json` - Expected outcomes for each hypothesis

3. Utilities

- `utils.py` - Helper function to load `hypotheses_outcomes.json`

4. Reference Material

- Medium article by George Fekkas - For understanding real-world threat hunting patterns used to build hypotheses. (reference only, do not hardcode detections from this article)
 - Link: [Reference Article](#)



Core Requirements

1. Query Generation System

Must implement:

- Query Generator
 - Generate executable queries from parsed hypotheses
 - Support your choice of interface:
 - SQL (DuckDB, SQLite, PostgreSQL)
 - Pandas DataFrame operations
 - Polars operations
 - Any other queryable interface

2. Evaluation Framework

3. Iteration & Improvement

Show your work:

- Document your initial approach and baseline scores
- Identify failure patterns from initial evaluation
- Describe improvements made (prompt engineering, validation logic, error handling)
- Show before/after metrics demonstrating improvement
- Discuss remaining limitations and trade-offs

4. Explainability

For each generated query, provide:

- Hypothesis interpretation: "This hypothesis is asking for..."
- Query reasoning: "I structured the query this way because..."
- Assumptions made: "I assumed 'recent' means last 7 days because..."
- Confidence score: "I'm 85% confident this query is correct because..."



Deliverables

Required Submissions

1. Code Repository with:

- `query_generator.py` or equivalent - Core query generation logic
- `evaluator.py` - Evaluation framework implementation
- `main.py` - Entry point to run full evaluation
- `requirements.txt` or `pyproject.toml` - Dependencies
- Unit tests for key components (optional but encouraged)

2. Documentation:

- `README.md` with:
 - Setup instructions (how to install and run)
 - Architecture overview (with diagram)
 - Design decisions and trade-offs
 - How to extend to other datasets
- `APPROACH.md` with:
 - Your prompting strategy (if using LLMs)
 - Iteration process and improvements made
 - Challenges faced and solutions
 - Limitations and future work

3. Evaluation Report:

- `evaluation_results.json` - Full evaluation output
- `EVALUATION_REPORT.md` with:
 - Overall metrics and scores



- Per-hypothesis breakdown
- Before/after metrics showing improvement

Optional (Bonus Points)

- Interactive Demo: Jupyter notebook or simple web UI (Streamlit) showing query generation in action
- Containerization: Dockerfile and docker-compose for easy setup
- Advanced Features:
 - Query optimization suggestions
 - Multi-step reasoning (generate multiple queries for complex hypotheses)
 - Confidence scoring with explanations
 - Automated prompt improvement based on failures
- Extended Evaluation:
 - Comparison of multiple prompting strategies
 - A/B testing different LLM models
 - Performance benchmarks (latency, throughput)



Submission Checklist

Before submitting, ensure you have:

- ☐ Code runs without errors on provided datasets
- ☐ All required files are included (query_generator, evaluator, main, README)
- ☐ Evaluation results are generated and included
- ☐ README has clear setup instructions
- ☐ Evaluation report discusses failures and improvements
- ☐ Code is reasonably clean and commented
- ☐ Dependencies are documented

Questions?

If you have questions about:

- CloudTrail schema: Refer to AWS CloudTrail documentation
- Expected outcomes format: Check the provided [hypotheses_outcomes.json](#)
- Evaluation metrics: Propose your own and justify them
- Technical approach: You have full freedom - choose what works best

Good luck! We're excited to see your approach to this challenging AI engineering problem.